

Project Milestone 1

CMPE 172

Project Name: Shuttleflow

Scenario: Shuttleflow is a badminton court and coaching booking system. Users are allowed to browse court availability, court times, coaching sessions, and manage their bookings. Coaches and managers can post availability.

Statements: This system is built with Java, Springboot, and SQL via JDBC (no ORM), using PostgreSQL as the database. This project realizes classic enterprise standards on the modern stack: Spring Boot in place of J2EE, and REST in place of CORBA/distributed objects.

User Roles

Role	Who	What can they do?
Customer	Player	- Browse open court times - Book or cancel court bookings
Admin	Court Manager/ Coach	- Manage court availability - View bookings - Create/remove time slots

Core Features

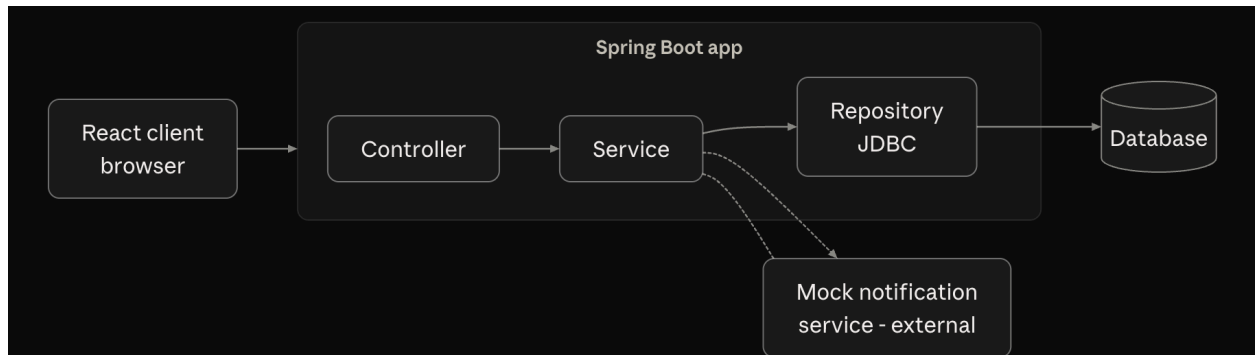
Features
Players log in with email and password
Users are able to filter by coach openings, session type (open play/coaching), and paginated search (10 per page) via SQL LIMIT + OFFSET)
Book a slot for open play
View upcoming confirmed bookings and history

Users can cancel bookings that they already booked.
Coach/ Court managers log in
Court Managers can set or remove court availability
Coaches can view appointments that are booked for them. (See who booked them)
Automated Email/SMS confirming that the user has reserved a booking at a certain time.
Logging/metric system showcasing users court booking activity and overall active history.
AI: Conversational booking assistant helping users book courts for play or coaching and available times.
Q&A section over a small internal knowledge base (RAG).

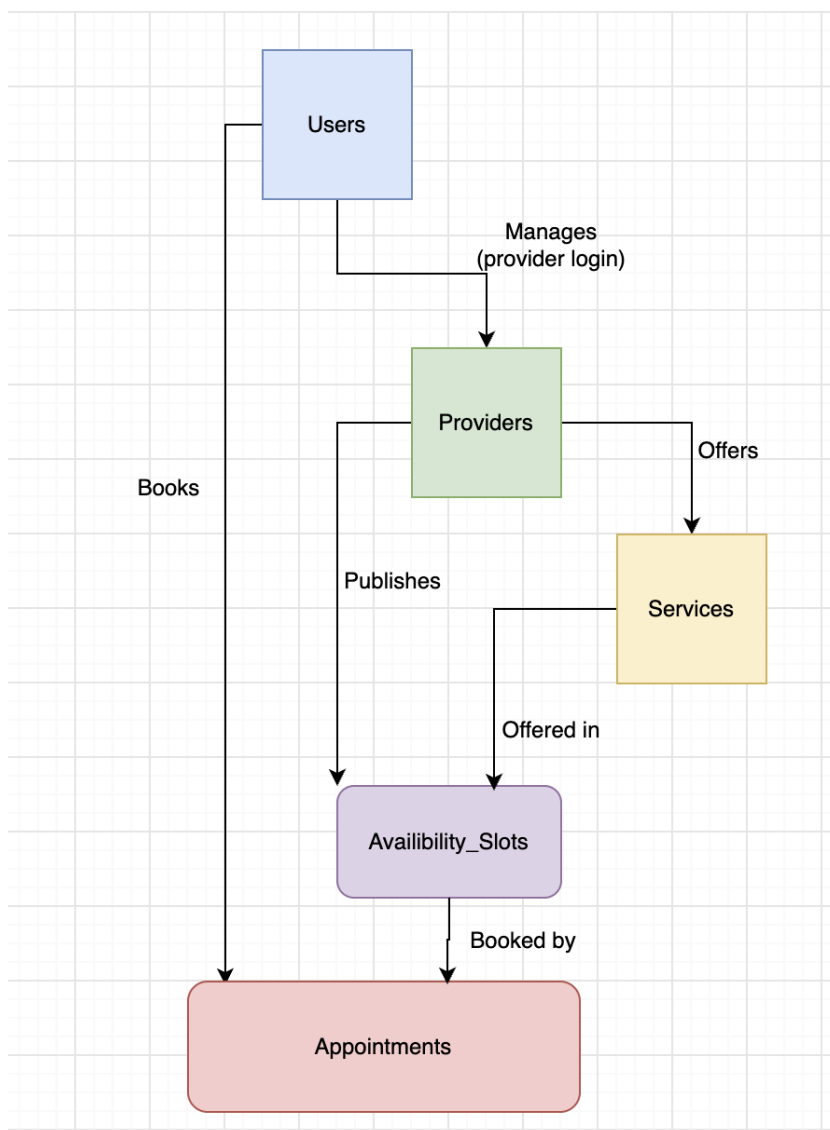
Main Entities

Entity	Who they are	Example
users	Players and staff accounts.	[Alex, role = CUSTOMER]
providers	A court or coach offering time.	[Court 3 / Coach Alex] OR [Court 3 / Open Play]
services	Session type a provider offers.	[Open Play, 60 minutes]
availability_slots	A bookable time slot for a provider + service.	[Court 3, 6-7 pm, Open Play]
appointments	A player's booking of a slot.	[Alex booked slot #4]

Block Diagram



ER Diagram:



Relational Schema

Table	Columns	Keys & Constraints
Users	user_id, password_hash, email, full_name, role, created_at	PK user_id, UNIQUE id, role IN ('CUSTOMER', 'PROVIDER')
Providers	provider_id, user_id, name, type, location	PK provider_id, FK user_id -> users, type IN ('COURT', 'COACH')
Services	service_id, provider_id, name, duration_min, price	PK service_id, FK provider_id -> providers, duration_min > 0,
Available_Slots	slot_id, provider_id, service_id, start_time, end_time, status	PK slot_id, FKs -> providers, services, end_time > start_time
Appointments	appointment_id, user_id, slot_id, status, booked_at	PK appointment_id, FK -> slots, users, UNIQUE (slot_id)

Keys and Relationships

Relationship	FK	Cardinality	English Explanation
Provider -> Service	service.provider_id	1:N	A coach can offer many sessions.
Provider -> Slot	availability_slot.provider_id	1:N	A coach can have many different time blocks.
Service -> Slot	availability_slot.service_id	1:N	A session type can be offered in many different time slots.
User -> Appointment	appointments.user_id	1:N	A player can book many slots.

Slot -> Appointment	appointments.slot_id	1:0..1	A slot is booked at most once.
User -> Provider	provider.user_id	1:0..1	Each user is at most, one provider, if a provider.

Double Booking Guard

Constraint: UNIQUE (slot_id) on appointments

- This stops two players from booking a court at the same time even if both were to press 'Book' at the same second. This is enforced now through java, but through the database. If two actions raced at the same time, the second insertion would fail through a unique constraint and would be caught by the service layer, returning "Court is already booked."

Justification

Cardinality: Shown in Keys and Relationships table.

Weak Entities: None currently. Every entity uses it's own ID but some are still based on the existence of other entities, but still require their own ID so are still considered strong.

Constraints: CHECK start_time > end_time; NOT NULL on FKs